EWB CLOUD LABS · STUDENT PROJECT · AWS ARCHITECT ASSOCIATE · PROJECT 01

AWS VPC Peering

Connect two VPCs. Prove private connectivity.

Student name	Sayantika Ghosh
Batch	Btech
Project title	AWS VPC Peering: Connect two VPCs, prove private connectivity
Region	us-east-1 (US East, N. Virginia)
Completion date	30-10-2026

All results in this report are the student's own execution results from the deployed environment. Where evidence was not captured, the report says so. No private keys, passwords or access tokens are included.

1. Summary

Two VPCs with non-overlapping CIDRs (`10.10.0.0/16` and `10.20.0.0/16`) were built in one AWS account and Region (`us-east-1`) and connected with the VPC peering connection `ewb-a-b-peer`. App-A (VPC A) and App-B (VPC B) have no public IPv4 addresses and exchanged ICMP traffic over their private IPs in both directions. Administration used one bastion in the public subnet of VPC A, reached from the laptop (WSL) with SSH ProxyJump. A controlled failure showed that removing App-B's ICMP rule stopped the ping and that restoring it recovered connectivity.

Test	Result
01 Baseline before peering	**Not recorded**
02 Forward path App-A → App-B	PASS, 4/4 replies, 0% packet loss
03 Reverse path App-B → App-A	PASS, 4/4 replies, 0% packet loss
04 Private SSH laptop → App-B via bastion	PASS
05 Isolation: no public IPv4 on App-A or App-B	PASS
Controlled failure (ICMP rule removed)	Ping blocked, 100% packet loss (as expected)
Recovery (ICMP rule restored)	PASS, 4/4 replies, 0% packet loss

2. Architecture and resource inventory

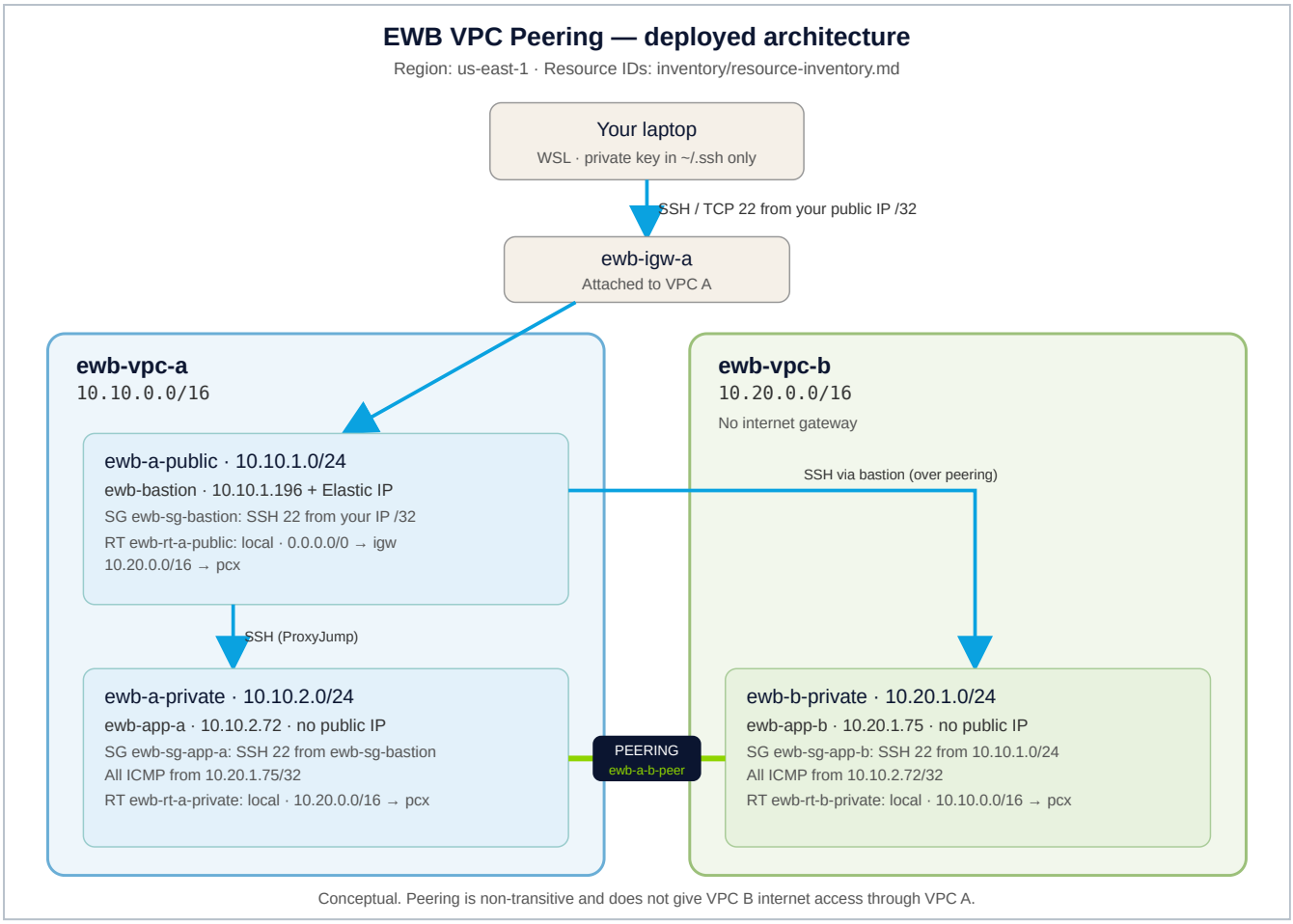


Figure: deployed architecture (`diagrams/architecture.svg`). Administration path: laptop → bastion → App-A or App-B using SSH ProxyJump. Project traffic: App-A ↔ App-B over private IPs and the peering connection. The bastion opens a separate SSH connection to the destination.

Why these CIDRs

10.10.0.0/16 and 10.20.0.0/16 do not overlap, which VPC peering requires. Each /24 subnet sits inside its own VPC range.

Network resources [actual IDs]

Name tag	Type	CIDR	Actual ID
ewb-vpc-a	VPC	10.10.0.0/16	vpc-051fb20bc75c4d6c4
ewb-a-public	Subnet (VPC A, us-east-1a)	10.10.1.0/24	subnet-0f0b9ba3ccae32a07
ewb-a-private	Subnet (VPC A, us-east-1a)	10.10.2.0/24	subnet-0f6af2b86cf58dc0c
ewb-vpc-b	VPC	10.20.0.0/16	vpc-060887d515a935fba
ewb-b-private	Subnet (VPC B)	10.20.1.0/24	subnet-02a6749cbddebed29
ewb-igw-a	Internet gateway (VPC A)	n/a	igw-0c0e6351e564526df
ewb-a-b-peer	Peering connection (A requester, B accepter)	n/a	pcx-0f1730cc1e9523713
ewb-rt-a-public	Route table	n/a	rtb-0d4f943ab6ceaf4e6
ewb-rt-a-private	Route table	n/a	rtb-00d4303fa8a4a5ea8
ewb-rt-b-private	Route table	n/a	Not captured
ewb-sg-bastion	Security group (VPC A)	n/a	sg-036a936bf7abdb647
ewb-sg-app-a	Security group (VPC A)	n/a	sg-049ac207ec5382f6c
ewb-sg-app-b	Security group (VPC B)	n/a	sg-03cde590d8254cf1d

Instances [actual]

Name tag	Subnet	Instance ID	Planned private IP	Actual private IP	Public IPv4
ewb-bastion	ewb-a-public	i-0e82bea59f7443a3f	10.10.1.10	10.10.1.196	Elastic IP 52.54.130.241
ewb-app-a	ewb-a-private	i-047cc151fe697b405	10.10.2.10	10.10.2.72	None
ewb-app-b	ewb-b-private	i-0a0bbfd5ce64d267e	10.20.1.10	10.20.1.75	None

All three: Amazon Linux 2023, `t3.micro`, key pair `ewb-lab-key-us`. Private IPs were assigned by AWS, so every test and security group source uses the actual addresses above.

3. Subnets, internet gateway and route tables

Route table	Association	Destination	Target	Evidence
ewb-rt-a-public	ewb-a-public	10.10.0.0/16 0.0.0.0/0 10.20.0.0/16	local ewb-igw-a pcx-0f1730cc1e9523713	Figure 4
ewb-rt-a-private	ewb-a-private	10.10.0.0/16 10.20.0.0/16	local pcx-0f1730cc1e9523713	Figure 5
ewb-rt-b-private	ewb-b-private	10.20.0.0/16 10.10.0.0/16	local pcx-0f1730cc1e9523713	Not captured

The public subnet needs `10.20.0.0/16 → pcx` because ProxyJump makes the bastion open the connection to App-B; this more specific route is selected instead of `0.0.0.0/0 → igw`. VPC B has no internet gateway.

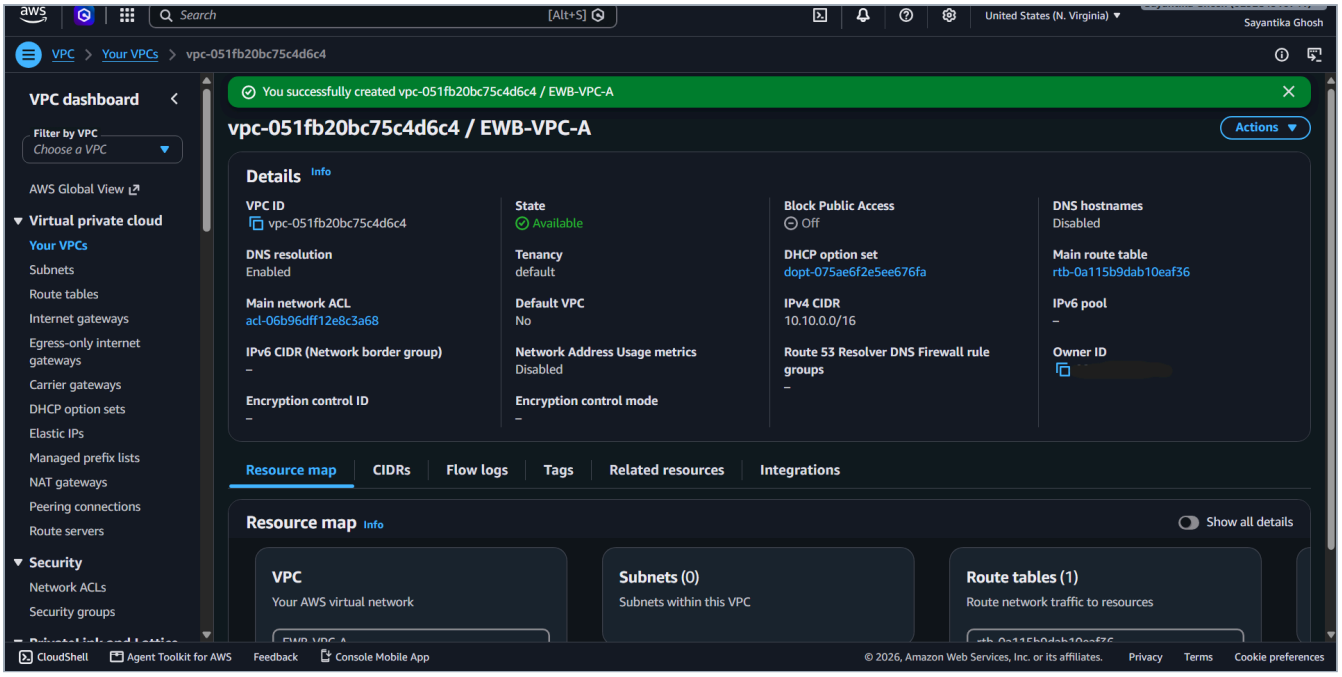


Figure 1. Host/resource: VPC console, ewb-vpc-a | Test/check: VPC A created | Result: vpc-051fb20bc75c4d6c4, 10.10.0.0/16, available

`evidence/screenshots/infrastructure/01-vpc-a-created.png`

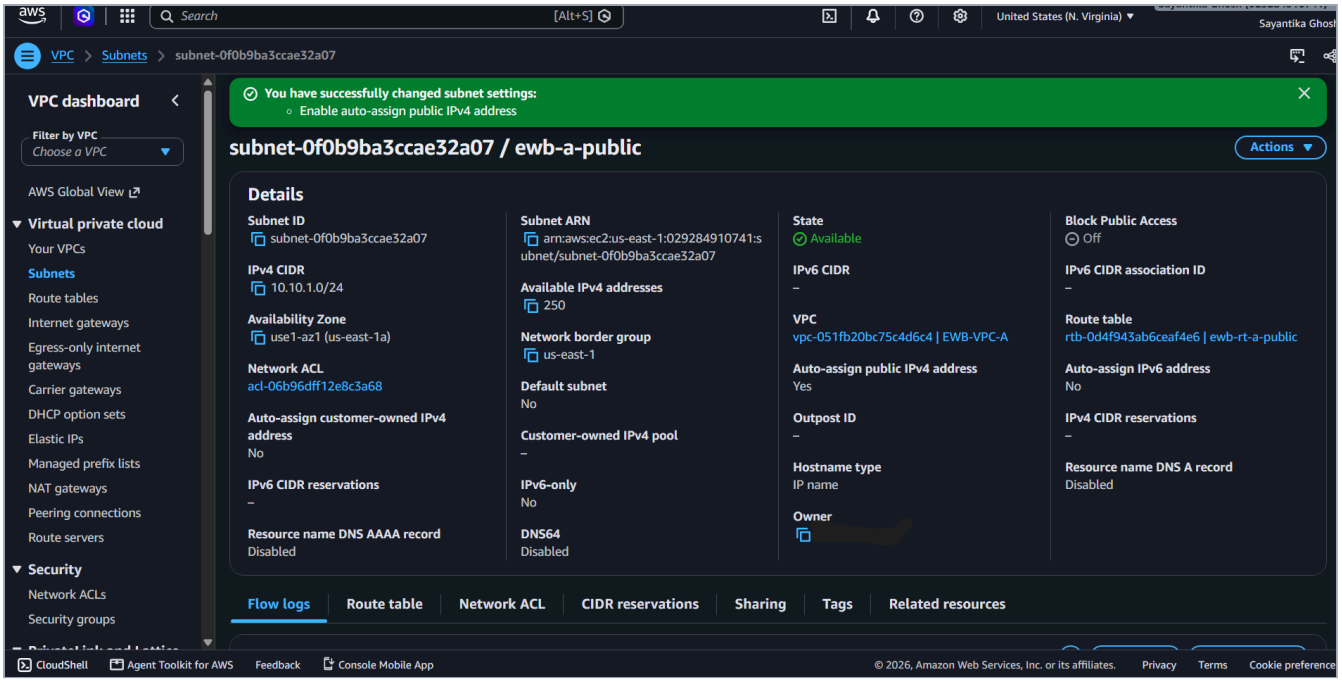


Figure 2. Host/resource: VPC console, ewb-a-public | Test/check: Public subnet details | Result: 10.10.1.0/24, us-east-1a, route table ewb-rt-a-public

`evidence/screenshots/infrastructure/02-subnet-a-public-details.png`

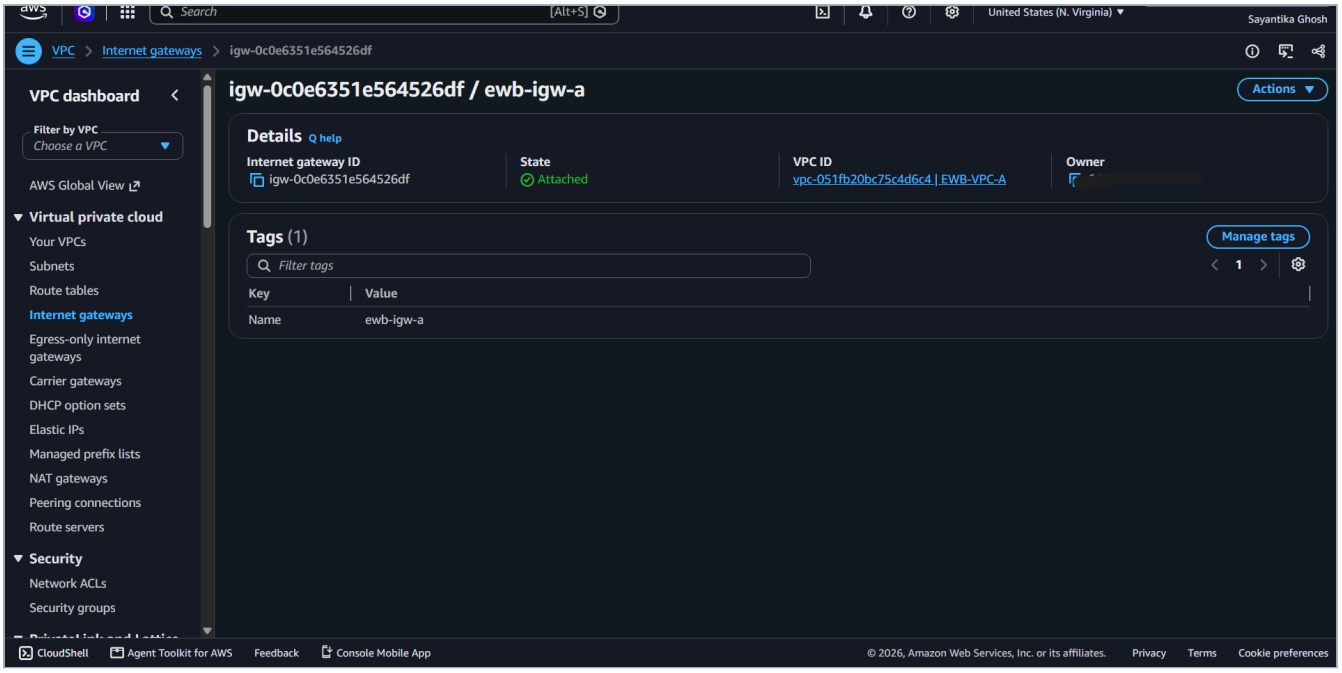


Figure 3. Host/resource: VPC console, ewb-igw-a | Test/check: Internet gateway attachment | Result: Attached to ewb-vpc-a

`evidence/screenshots/infrastructure/03-igw-a-attached.png`

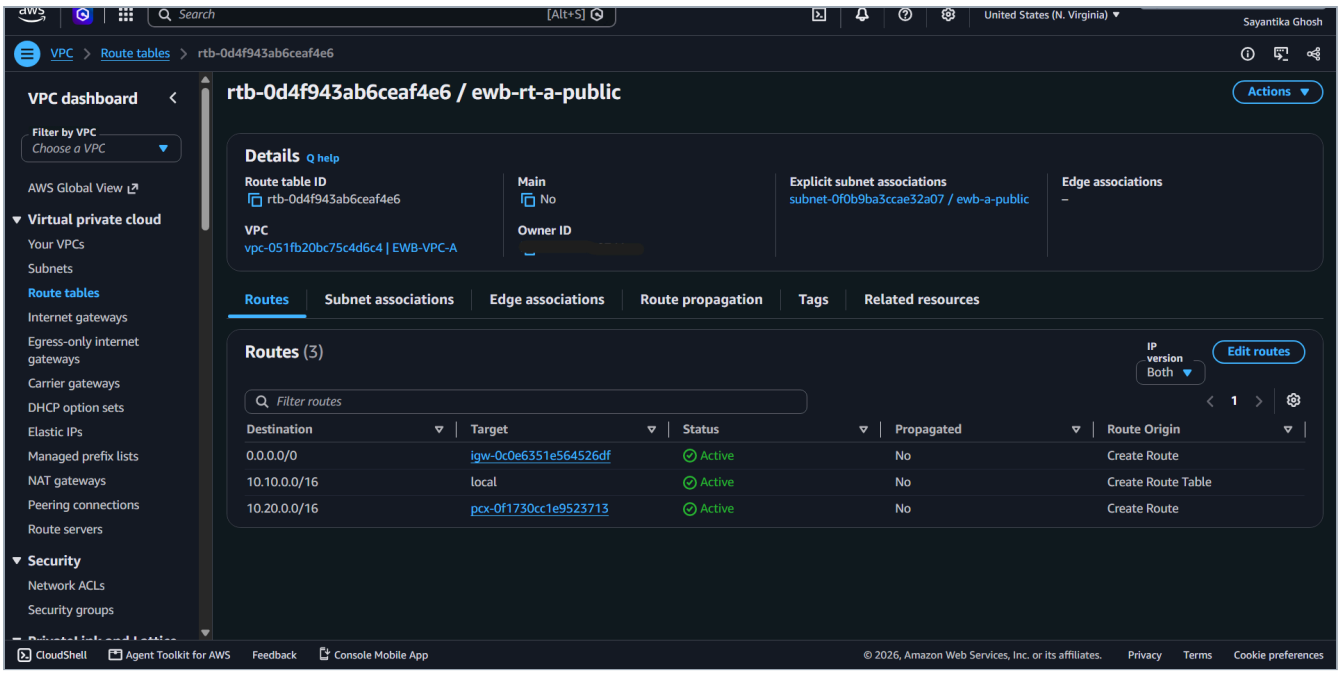


Figure 4. Host/resource: VPC console, ewb-rt-a-public | Test/check: Checkpoint 01/04: routes and association | Result: local, 0.0.0.0/0 → igw, 10.20.0.0/16 → pcx; associated with ewb-a-public
`evidence/screenshots/infrastructure/04-rt-a-public-routes-association.png`

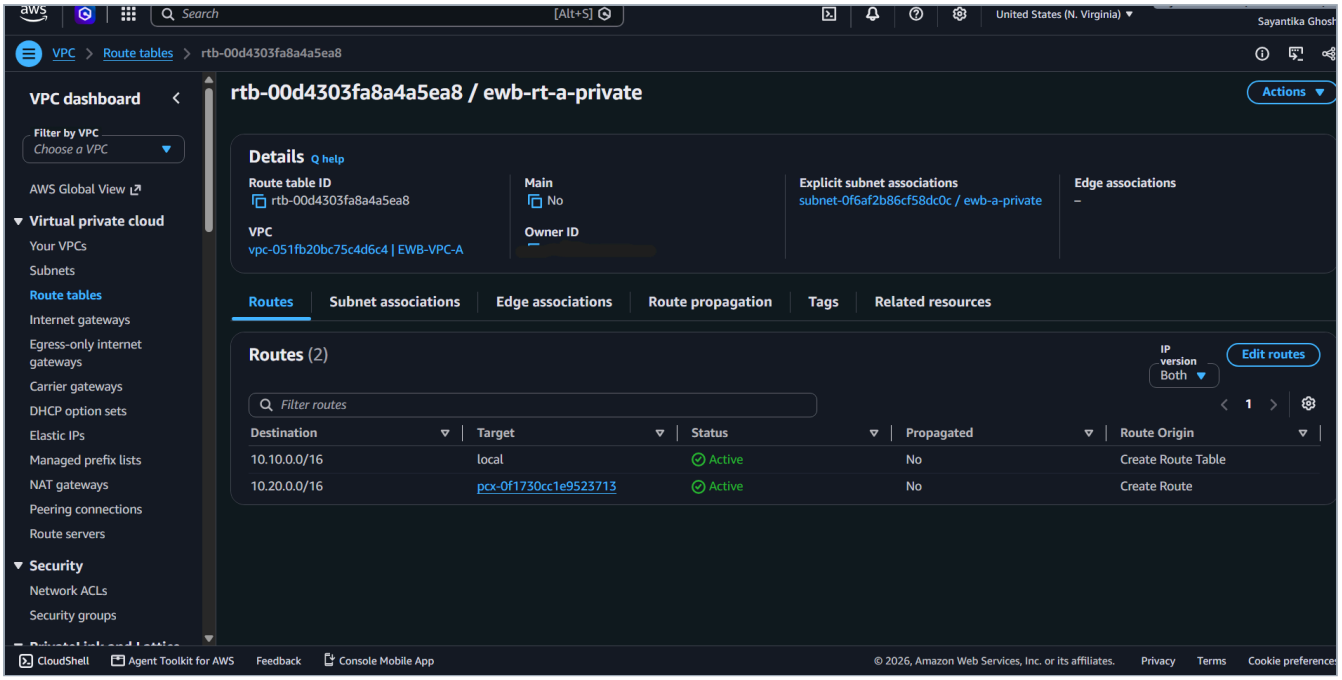


Figure 5. Host/resource: VPC console, ewb-rt-a-private | Test/check: Checkpoint 04: routes and association | Result: local, 10.20.0.0/16 → pcx (Active); associated with ewb-a-private
`evidence/screenshots/infrastructure/05-rt-a-private-routes-association.png`

4. Peering connection and forward / return routes

The peering connection `ewb-a-b-peer` (`pcx-0f1730cc1e9523713`) was requested from VPC A (requester) to VPC B (accepter) in the same account and Region, then accepted. The screenshot below shows the request before acceptance. A screenshot of the connection in Active state was not captured; the peering route in Figure 5 shows status Active, and tests 02 to 04 could only succeed over an active connection.

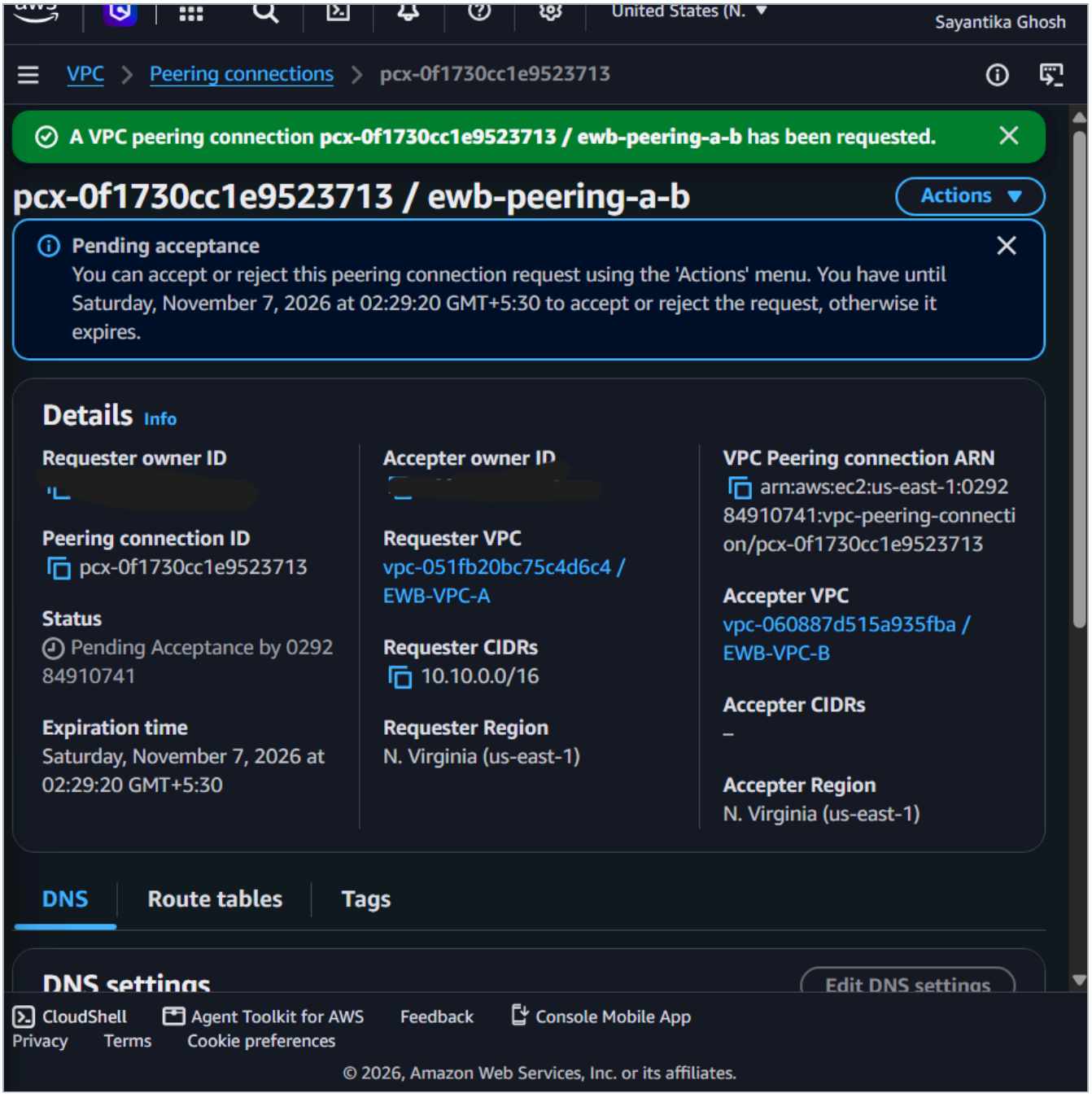


Figure 6. Host/resource: VPC console, ewb-a-b-peer | Test/check: Peering request created | Result: Requester ewb-vpc-a, accepter ewb-vpc-b, Pending acceptance (before accept)
`evidence/screenshots/infrastructure/06-peering-request-pending.png`

Forward path (App-A → App-B)

App-A `10.10.2.72` → `ewb-rt-a-private` route `10.20.0.0/16 → pcx` → VPC B → `ewb-sg-app-b` allows ICMP from `10.10.2.72/32` → App-B `10.20.1.75`.

Return path (App-B → App-A)

App-B's reply uses `ewb-rt-b-private` route `10.10.0.0/16 → pcx` back to App-A. Security groups are stateful, so the reply is allowed automatically, but without the return route the reply would have no path back. A one-way route is not enough.

Peering is one-to-one and non-transitive, and it does not give VPC B internet access through VPC A's internet gateway.

5. Security groups and private SSH access

Security group	Rule	Workbook source	Deployed source
ewb-sg-bastion	SSH 22	Your public IPv4 /32	Student public IPv4 /32 (as captured at launch)
ewb-sg-app-a	SSH 22	10.10.1.10/32 (bastion)	Security group ewb-sg-bastion
ewb-sg-app-a	ICMP	Echo request from 10.20.1.10/32	All ICMP IPv4 from 10.20.1.75/32 (App-B)
ewb-sg-app-b	SSH 22	10.10.1.10/32 (bastion)	10.10.1.0/24 (public subnet A)
ewb-sg-app-b	ICMP	Echo request from 10.10.2.10/32	All ICMP IPv4 from 10.10.2.72/32 (App-A)

Default outbound allow-all rules and default network ACLs were kept. The deployed rules are broader than the workbook's (All ICMP instead of Echo request; a /24 SSH source on App-B; a security group reference on App-A), but nothing is open to `0.0.0.0/0`. App-B's SSH source must cover the bastion's private IP (`10.10.1.196`) because ProxyJump makes the bastion open the connection.

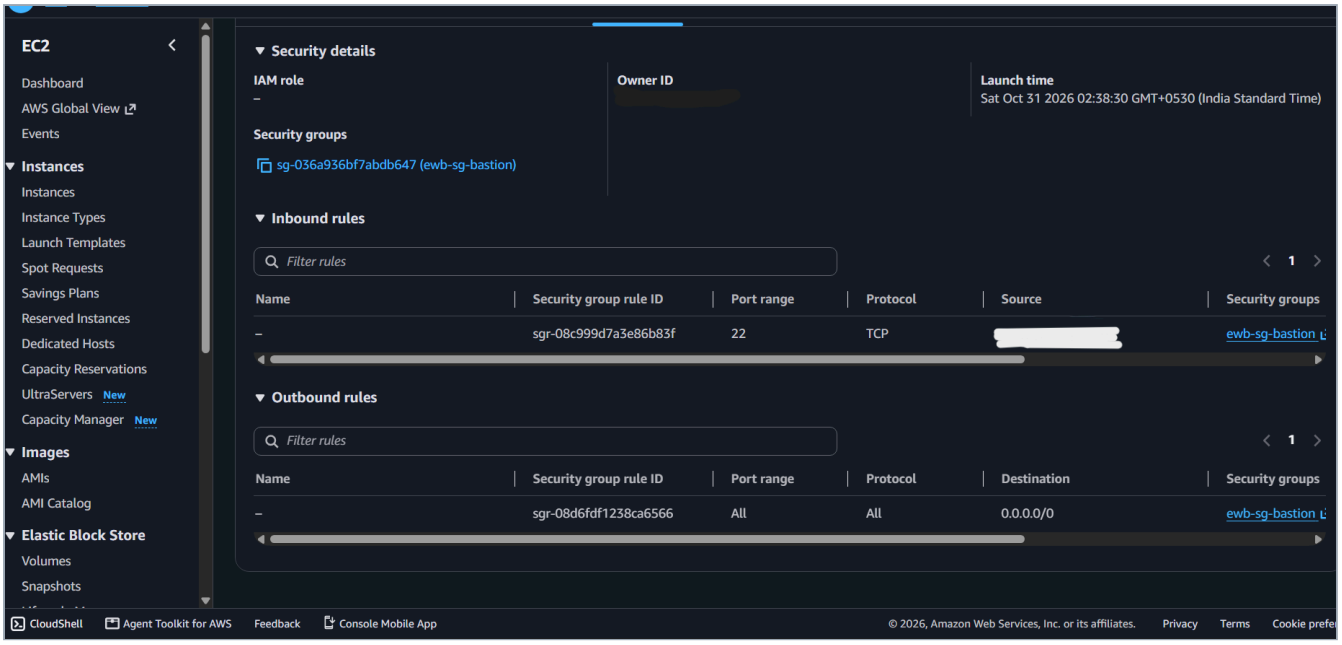


Figure 7. Host/resource: EC2 console, ewb-bastion security tab | Test/check: Bastion SG inbound rules | Result: SSH 22 from the student's public IPv4 /32

`evidence/screenshots/security/01-sg-bastion-inbound-rules.png`

The bastion group was edited later; its final rule set was not captured.

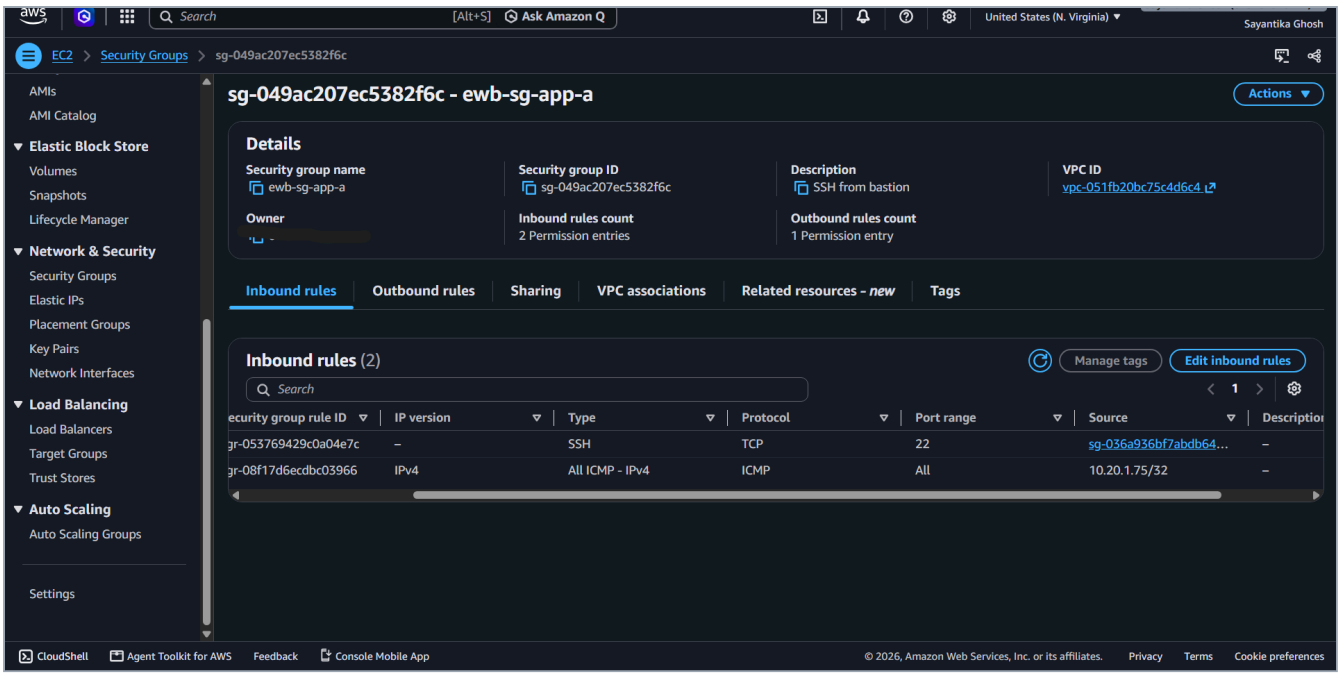


Figure 8. Host/resource: EC2 console, ewb-sg-app-a | Test/check: App-A SG inbound rules | Result: SSH from ewb-sg-bastion; All ICMP from 10.20.1.75/32

`evidence/screenshots/security/02-sg-app-a-inbound-rules.png`

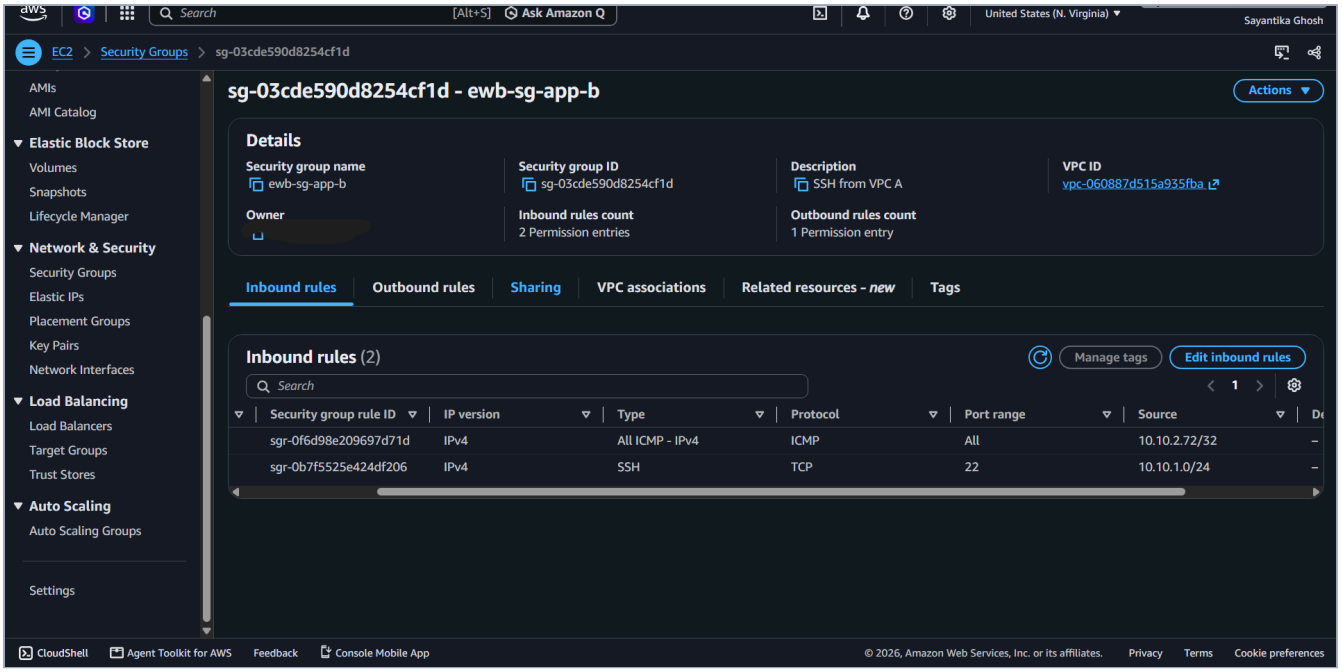


Figure 9. Host/resource: EC2 console, ewb-sg-app-b | Test/check: App-B SG inbound rules | Result: All ICMP from 10.10.2.72/32; SSH from 10.10.1.0/24

`evidence/screenshots/security/03-sg-app-b-inbound-rules.png`

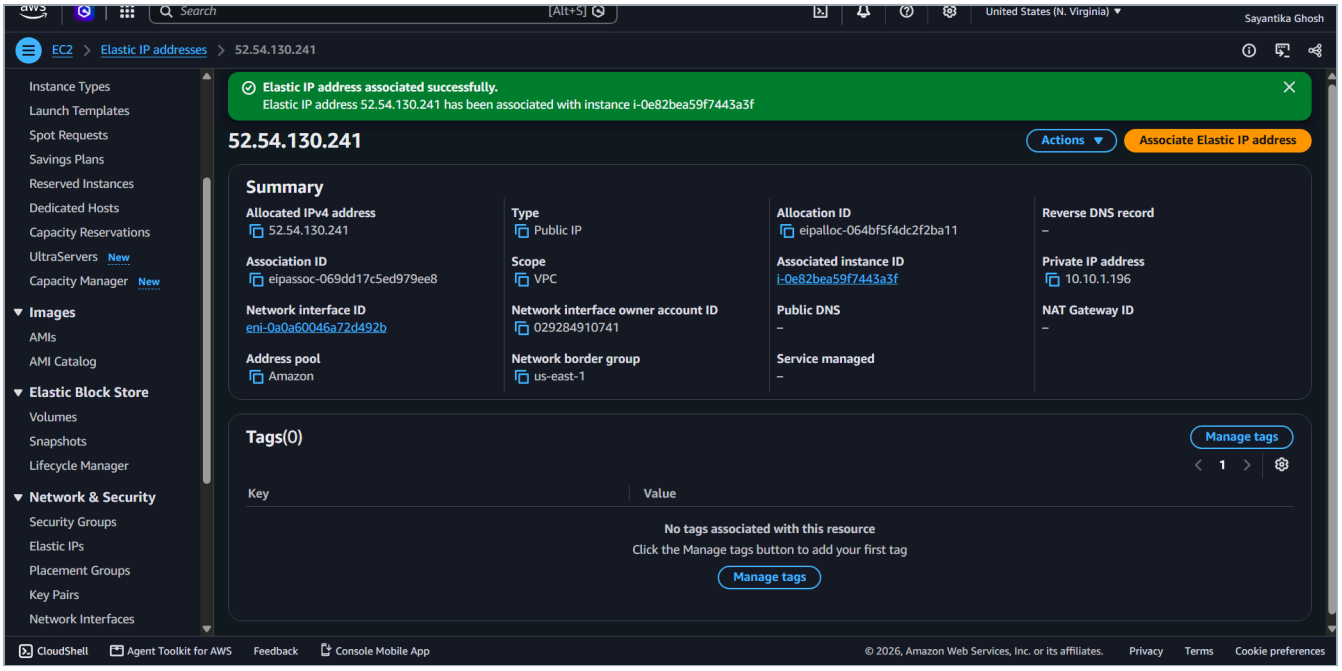


Figure 10. Host/resource: EC2 console, Elastic IPs | Test/check: Bastion public address | Result: 52.54.130.241 associated with ewb-bastion (10.10.1.196)

`evidence/screenshots/infrastructure/09-bastion-elastic-ip-associated.png`

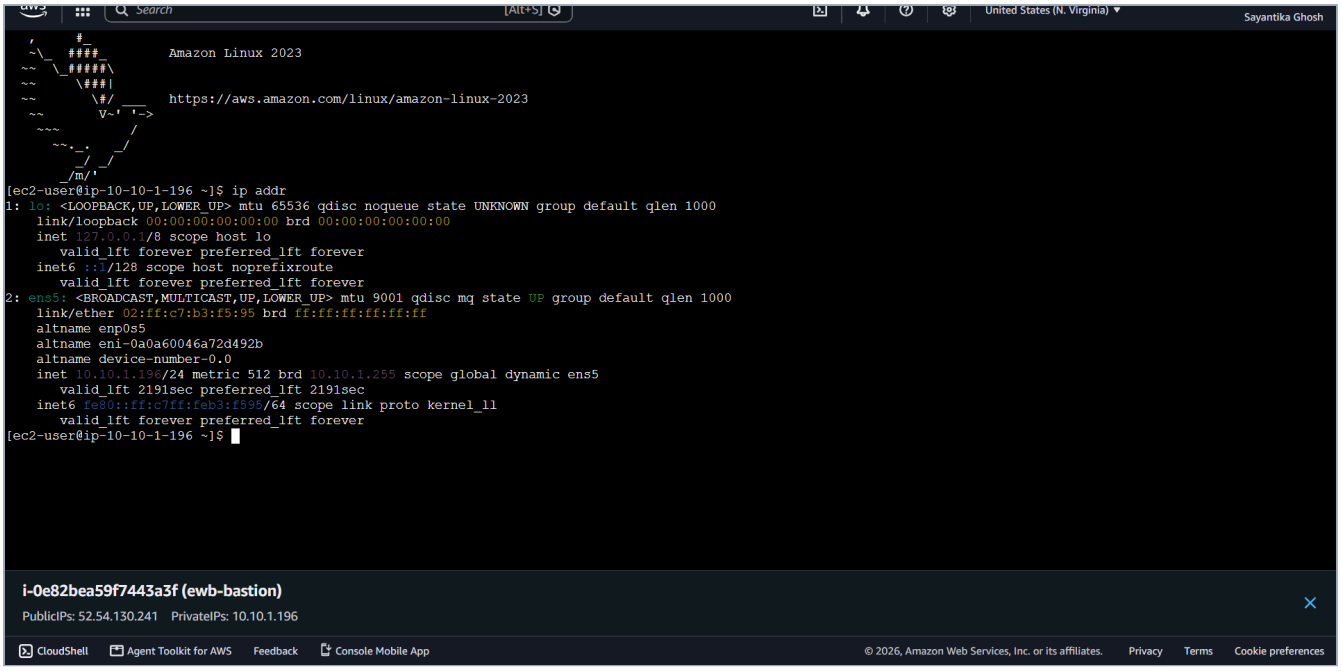


Figure 11. Host/resource: ewb-bastion (EC2 Instance Connect) | Test/check: Bastion identity | Result: i-0e82bea59f7443a3f, public 52.54.130.241, private 10.10.1.196

`evidence/screenshots/infrastructure/10-bastion-instance-connect-session.png`

6. Test evidence

> **Test 01 baseline was not recorded.** The ping from App-A to App-B before peering and routes was not captured, so no result is claimed. The expected result was no replies, because VPC A had no route to 10.20.0.0/16.

Tests 02 and 03: forward and reverse ping

Run from the laptop (WSL) through the bastion: `ssh ewb-app-a "ping -c 4 10.20.1.75"` and `ssh ewb-app-b "ping -c 4 10.10.2.72"`. The saved file names are swapped relative to the test numbers and are kept as recorded.

Test 02, App-A → App-B (file: ewb-test-03.txt). Result: 4 received, 0% packet loss — PASS

```
PING 10.20.1.75 (10.20.1.75) 56(84) bytes of data.
64 bytes from 10.20.1.75: icmp_seq=1 ttl=127 time=0.449 ms
64 bytes from 10.20.1.75: icmp_seq=2 ttl=127 time=0.438 ms
64 bytes from 10.20.1.75: icmp_seq=3 ttl=127 time=0.478 ms
64 bytes from 10.20.1.75: icmp_seq=4 ttl=127 time=49.9 ms

--- 10.20.1.75 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3101ms
rtt min/avg/max/mdev = 0.438/12.820/49.915/21.416 ms
```

Test 03, App-B → App-A (file: ewb-test-02.txt). Result: 4 received, 0% packet loss — PASS

```
PING 10.10.2.72 (10.10.2.72) 56(84) bytes of data.
64 bytes from 10.10.2.72: icmp_seq=1 ttl=127 time=0.452 ms
64 bytes from 10.10.2.72: icmp_seq=2 ttl=127 time=0.463 ms
64 bytes from 10.10.2.72: icmp_seq=3 ttl=127 time=0.456 ms
64 bytes from 10.10.2.72: icmp_seq=4 ttl=127 time=0.421 ms

--- 10.10.2.72 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3095ms
rtt min/avg/max/mdev = 0.421/0.448/0.463/0.016 ms
```

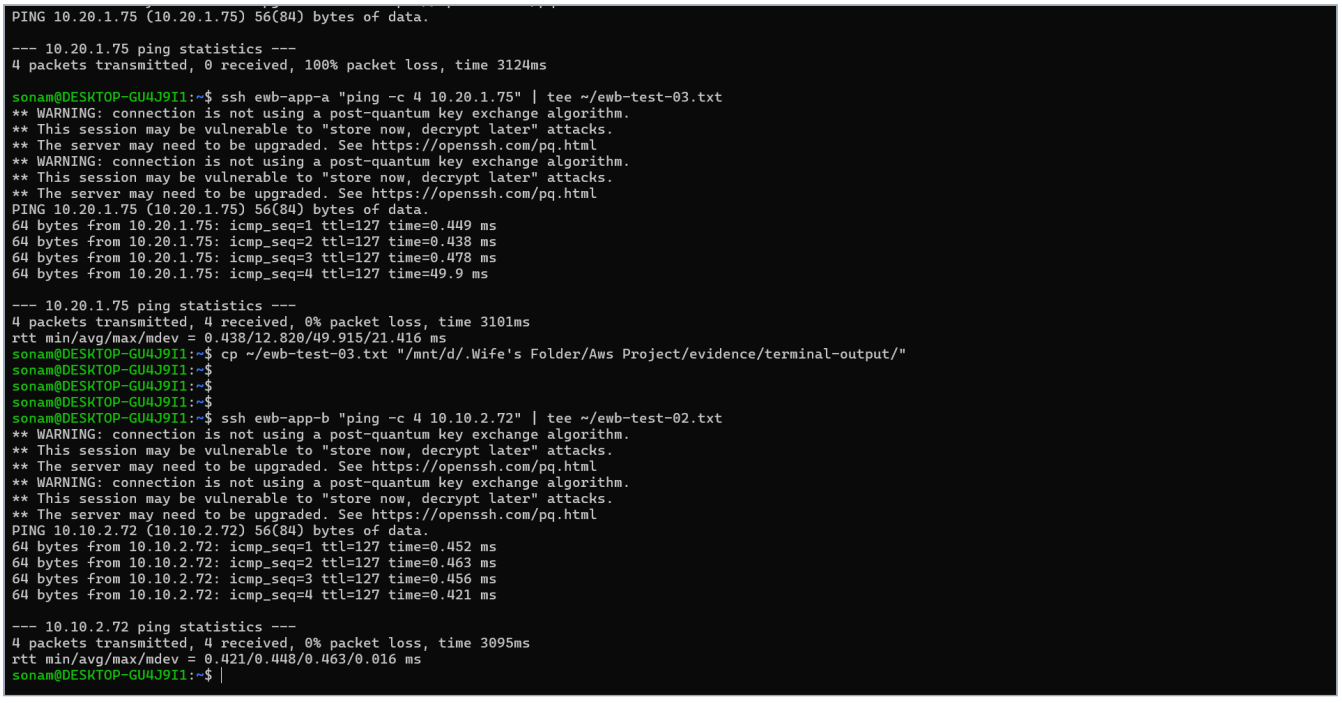


Figure 12. Host/resource: Laptop (WSL) → App-A and App-B | Test/check: Tests 02 and 03 | Result: Both directions 0% packet loss

`evidence/screenshots/tests/test-02-03-ping-forward-and-reverse.png`

Test 04: private SSH to App-B through the bastion

From the laptop (WSL): `ssh ewb-app-b` using the ProxyJump configuration in `config/ssh-config.example`, then `hostname` and `ip addr show ens5`.

Test 04, App-B identity (file: ewb-test-04.txt). Result: shell on App-B — PASS

```
ip-10-20-1-75.ec2.internal
2: ens5: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 9001 qdisc mq state UP group default qlen 1000
    link/ether 12:ff:c1:ca:f1:c1 brd ff:ff:ff:ff:ff:ff
    altname enp0s5
    altname eni-011fdfd6ccb9f4dcf
    altname device-number-0.0
    inet 10.20.1.75/24 metric 512 brd 10.20.1.255 scope global dynamic ens5
       valid_lft 2597sec preferred_lft 2597sec
    inet6 fe80::10ff:c1ff:feca:f1c1/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
```

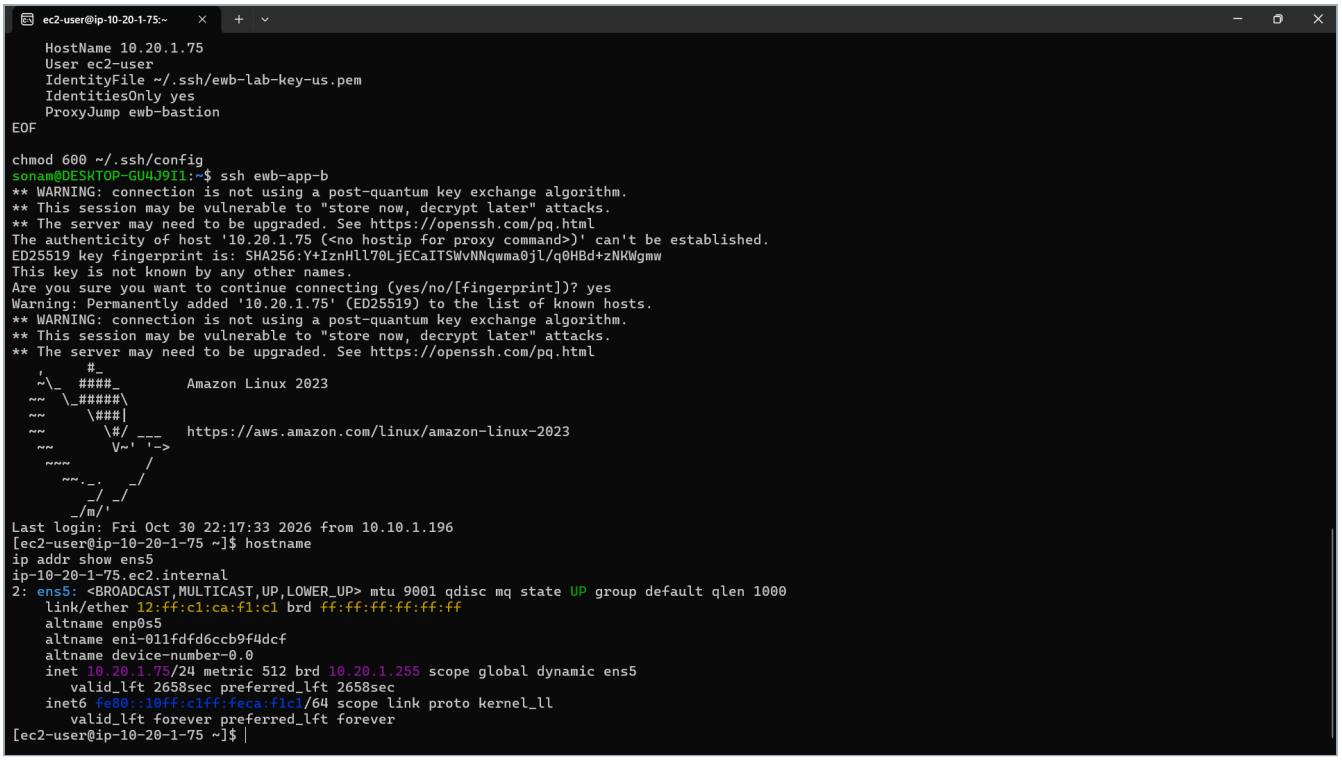


Figure 13. Host/resource: Laptop (WSL) → bastion → App-B | Test/check: Test 04 ProxyJump SSH | Result: Shell on ip-10-20-1-75.ec2.internal, ens5 10.20.1.75/24
`evidence/screenshots/tests/test-04-wsl-proxyjump-ssh-app-b.png`

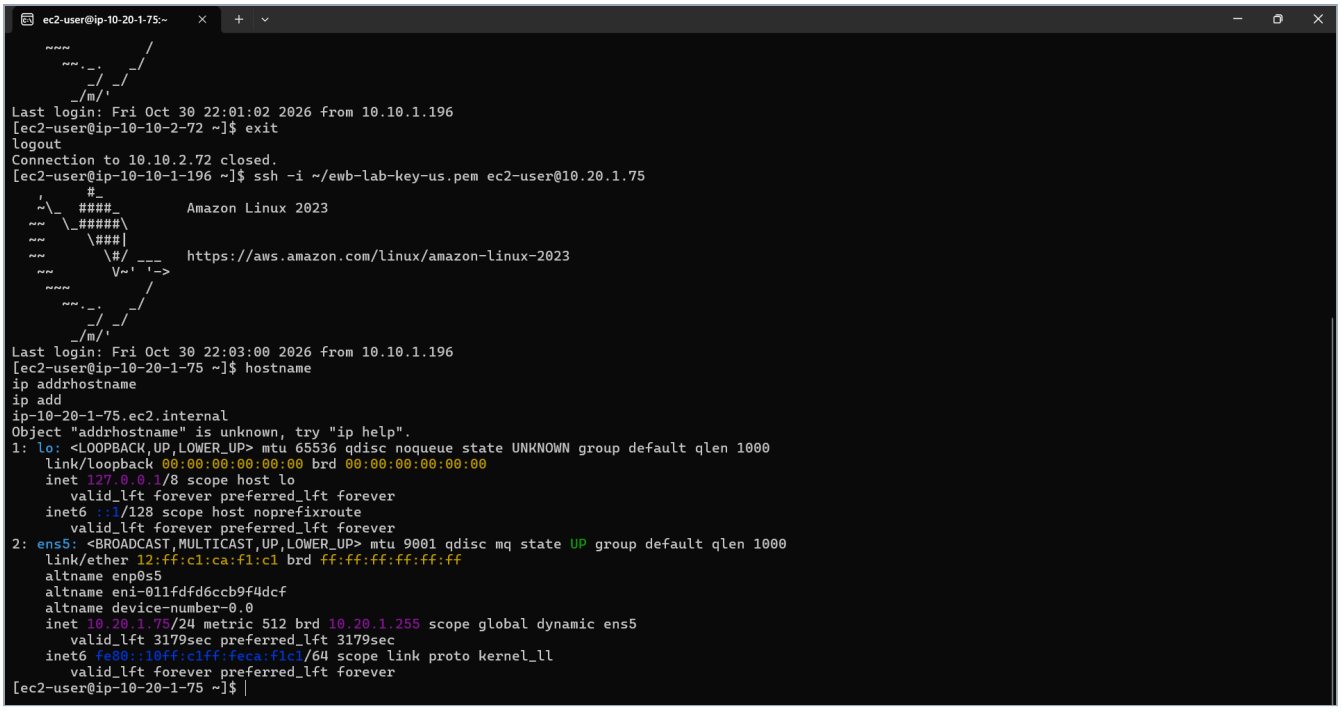


Figure 14. Host/resource: ewb-bastion (10.10.1.196) → App-B | Test/check: Earlier bastion-hop SSH (key on bastion, see section 8) | Result: Shell on App-B, 10.20.1.75/24

`evidence/screenshots/tests/test-04-bastion-ssh-app-b-hostname-ip.png`

Test 05: isolation

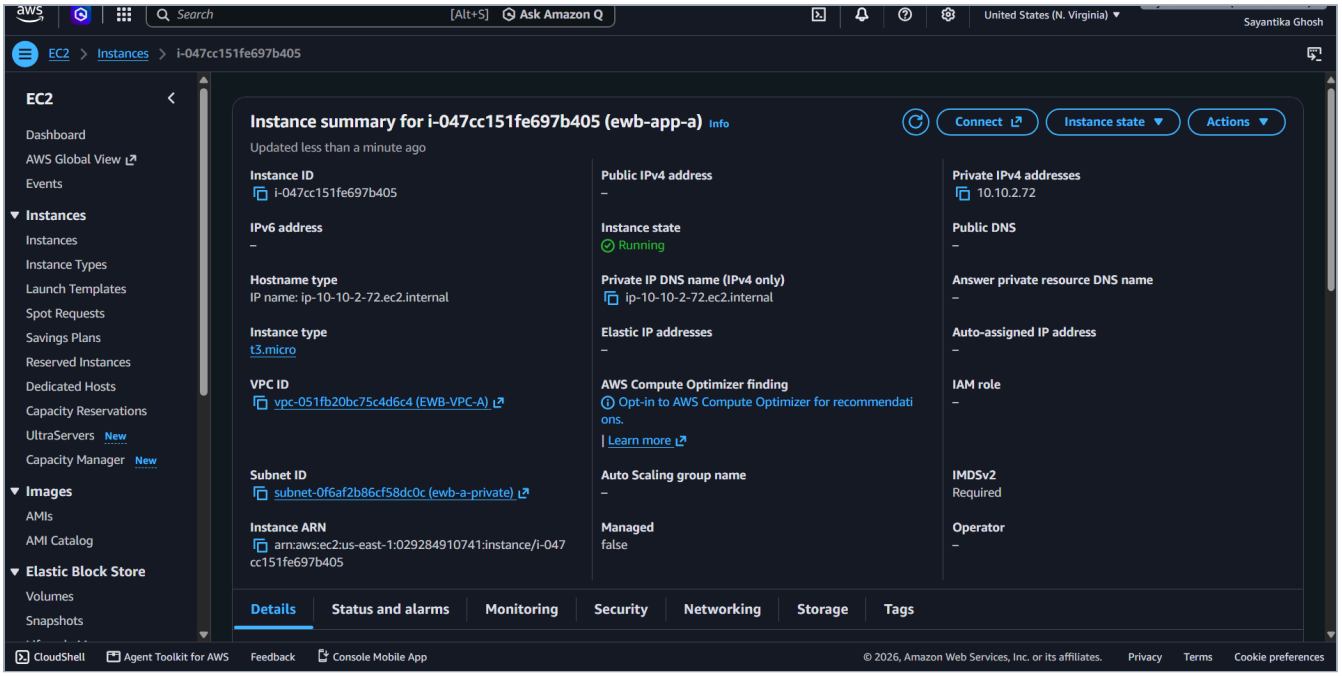


Figure 15. Host/resource: EC2 console, ewb-app-a | Test/check: Test 05 isolation | Result: Private 10.10.2.72, no public IPv4 — PASS

`evidence/screenshots/infrastructure/07-ec2-app-a-details-no-public-ip.png`

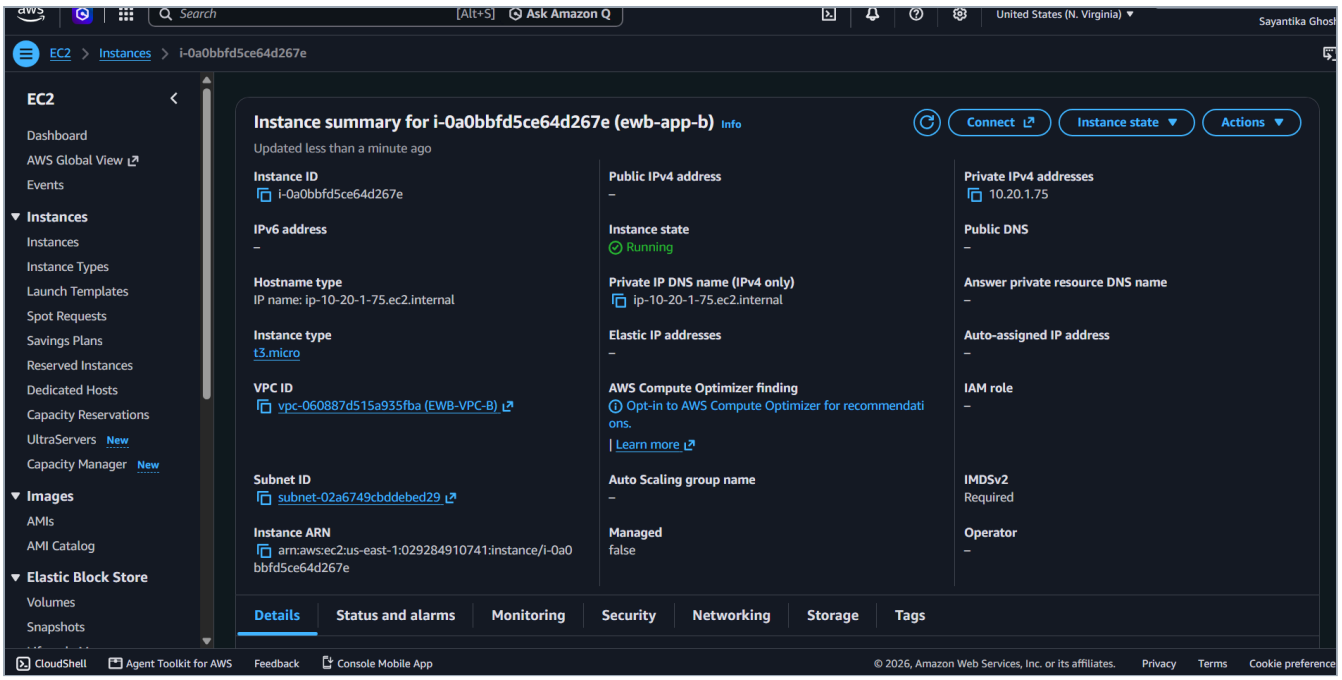


Figure 16. Host/resource: EC2 console, ewb-app-b | Test/check: Test 05 isolation | Result: Private 10.20.1.75, no public IPv4 — PASS

`evidence/screenshots/infrastructure/08-ec2-app-b-details-no-public-ip.png`

7. Controlled ICMP failure and recovery

Only App-B's inbound ICMP rule (source `10.10.2.72/32`) was removed; the SSH rule stayed. The forward ping from App-A was repeated, then the rule was restored and the ping repeated again.

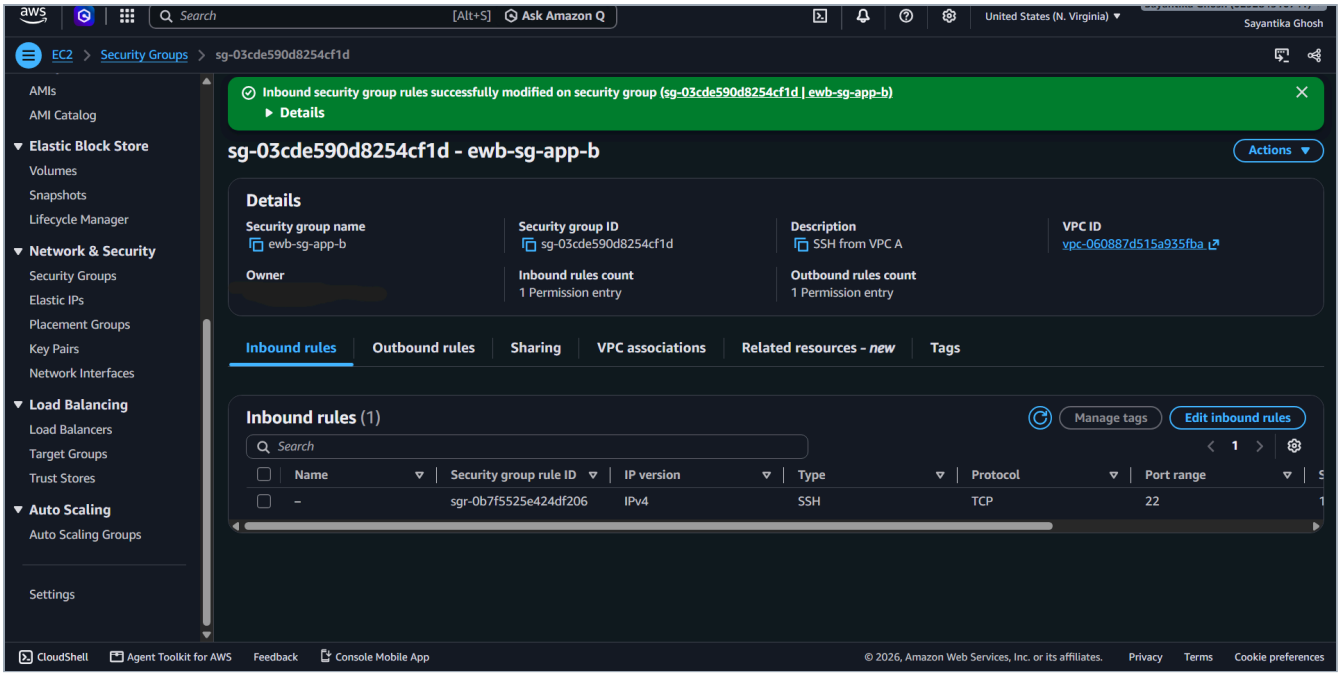


Figure 17. Host/resource: EC2 console, ewb-sg-app-b | Test/check: F1: ICMP rule removed | Result: One inbound rule left (SSH 22)
`evidence/screenshots/tests/test-f1-app-b-icmp-rule-removed.png`

F2, App-A → App-B after removal (file: ewb-f1.txt). Result: 0 received, 100% packet loss — failed as expected

```
PING 10.20.1.75 (10.20.1.75) 56(84) bytes of data.

--- 10.20.1.75 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3093ms
```

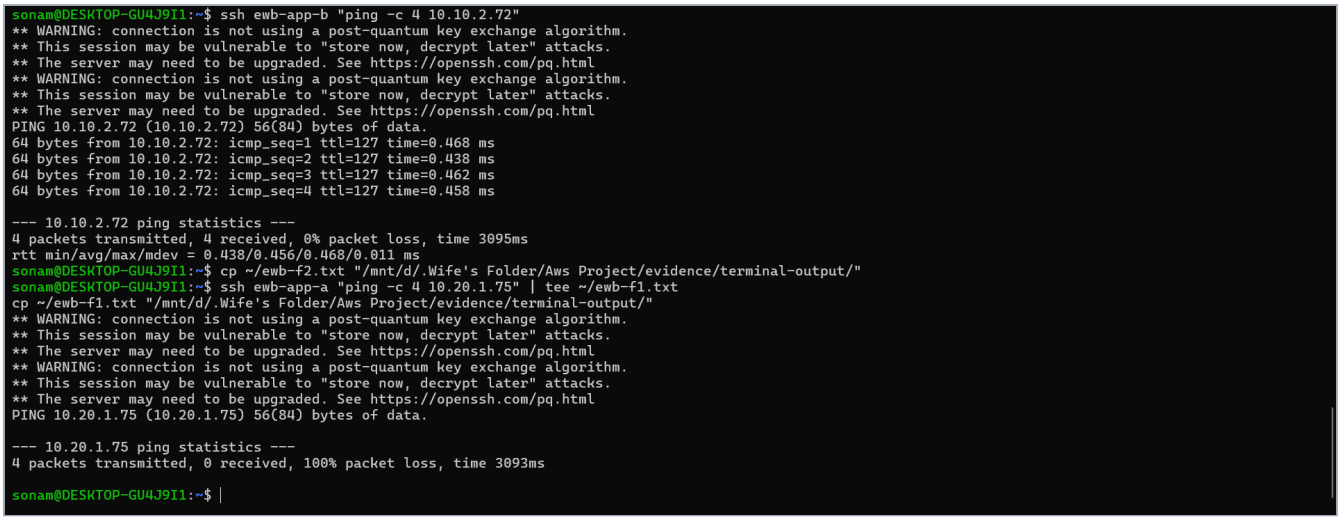


Figure 18. Host/resource: Laptop (WSL) → App-A | Test/check: F2: ping after rule removal | Result: 100% packet loss
`evidence/screenshots/tests/test-f2-ping-blocked-after-rule-removal.png`

F3, rule restored: no screenshot was captured. **F4, ping after restoring:**

F4, App-A → App-B after restoring the rule (file: ewb-f4.txt). Result: 4 received, 0% packet loss — recovered

```
PING 10.20.1.75 (10.20.1.75) 56(84) bytes of data.
64 bytes from 10.20.1.75: icmp_seq=1 ttl=127 time=0.445 ms
64 bytes from 10.20.1.75: icmp_seq=2 ttl=127 time=0.472 ms
64 bytes from 10.20.1.75: icmp_seq=3 ttl=127 time=0.455 ms
64 bytes from 10.20.1.75: icmp_seq=4 ttl=127 time=0.458 ms

--- 10.20.1.75 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3106ms
rtt min/avg/max/mdev = 0.445/0.457/0.472/0.009 ms
```

> **Explanation.** With the ICMP rule removed, App-B's security group dropped App-A's echo requests, so no replies came back even though the peering connection and both routes were unchanged. Restoring the rule let the requests in, and the stateful security group allowed the replies out. An active peering connection and correct routes are necessary but not sufficient: the destination security group must also allow the traffic.

8. Deviations and limitations

Item	Workbook	Actual
Region	ap-south-1 (example)	us-east-1. A first attempt in ap-south-1 was abandoned because the new account could launch EC2 only in US Regions at the time.
Private IPs	10.10.1.10, 10.10.2.10, 10.20.1.10	10.10.1.196, 10.10.2.72, 10.20.1.75 (AWS-assigned)
Key pair name	ewb-lab	ewb-lab-key-us
Security group rules	/32 sources, ICMP Echo request	Broader rules (section 5)
Private key	Never on an instance	Copied to the bastion for early SSH tests, then deleted (Figure below). Test 04 was run from the laptop with ProxyJump.
Bastion public IP	Auto-assigned public IPv4	Elastic IP 52.54.130.241
Availability Zone	Same AZ where available	VPC A subnets in us-east-1a; App-B in us-east-1b

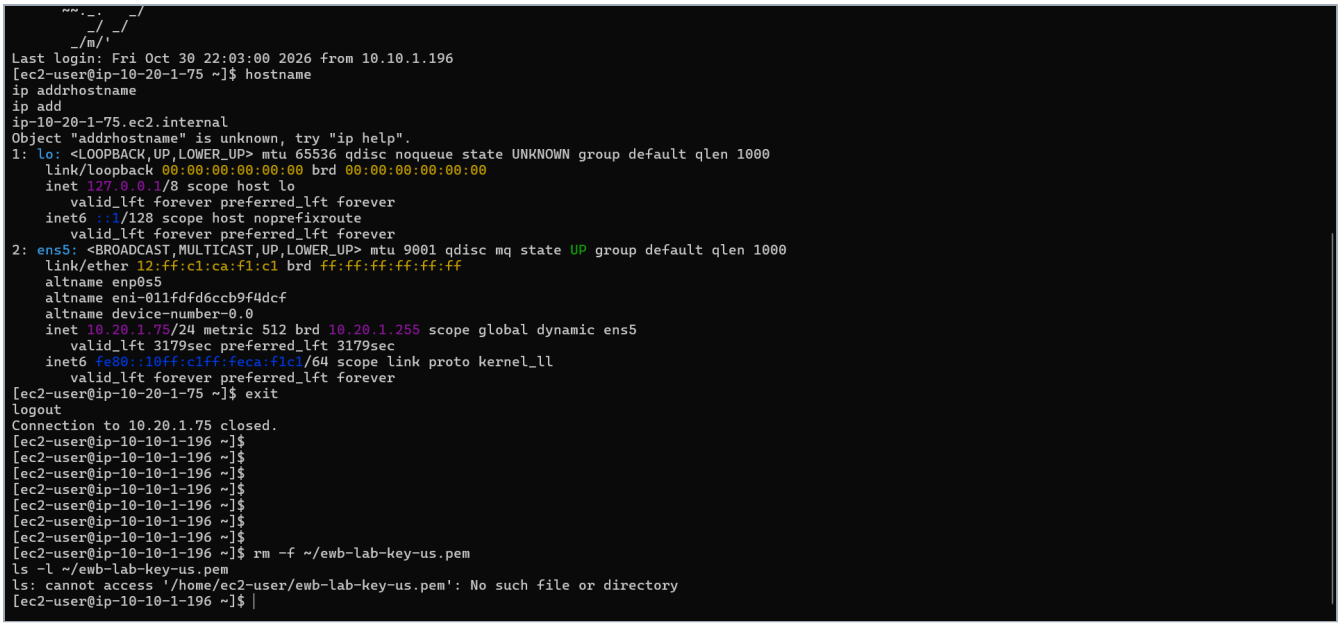


Figure 19. Host/resource: ewb-bastion | Test/check: Private key removed from the bastion | Result: rm -f ~/ewb-lab-key-us.pem; file no longer present
`evidence/screenshots/security/04-bastion-private-key-removed.png`

Evidence not captured

- Test 01 baseline.
- Peering connection in Active state with both VPC IDs (Checkpoint 03).
- VPC B, `ewb-b-private` and `ewb-rt-b-private` routes and association; the route table ID.
- Final `ewb-sg-bastion` rule set; F3 rule restore; F4 screenshot.
- The ping output files do not include `hostname`; the source host is identified from the command shown in the screenshots.

Issue met during the lab

Before test 02 was recorded, a forward ping from App-A failed with 100% packet loss because `ewb-sg-app-b` did not yet have an ICMP rule. The rule was added and the test was rerun successfully. This happened after peering, so it is not a substitute for the test 01 baseline.

9. Cleanup

> **Cleanup has not been performed at the time of this report, and no cleanup evidence is included.**

Planned order (workbook page 14; full checklist in `cleanup/cleanup-checklist.md`):

1. Terminate ewb-bastion, ewb-app-a and ewb-app-b; confirm their EBS volumes are deleted; release Elastic IP 52.54.130.241; remove the unused key pair and local key if no longer needed.
2. Delete the peer routes and ewb-a-b-peer; delete the lab security groups; delete the subnets and custom route tables; detach and delete ewb-igw-a; delete both VPCs.
3. Filter by `Project = EWB-VPC-Peering`, check EC2, volumes, public IPv4 and VPC resources in us-east-1, check Billing, and capture cleanup evidence.

10. What I learned

Through this project, I gained practical experience in creating and connecting two AWS VPCs using VPC Peering. I learned how to configure subnets, route tables, internet gateways and security groups, and how each component contributes to private network communication. Testing connectivity in both directions helped me understand the importance of forward and return routes. I also experienced how a security group rule can affect connectivity through the controlled ICMP failure and recovery test. Setting up SSH access through a bastion host using ProxyJump gave me a better understanding of securely accessing private EC2 instances without assigning them public IP addresses. Overall, this project helped me connect the networking concepts learned in theory with their practical implementation on AWS.

11. Conclusion

The project built two non-overlapping VPCs in us-east-1 and connected them with VPC peering. Tests 02 and 03 show two-way private connectivity between App-A and App-B, test 04 shows administrative SSH to App-B through the bastion with ProxyJump, and test 05 confirms that neither application server has a public IPv4 address. The controlled failure showed that the destination security group controls reachability independently of peering and routing. The pre-peering baseline (test 01) was not recorded, and the deviations from the workbook are listed in section 8.

Supporting files: README.md, inventory/resource-inventory.md, evidence/test-results.md, evidence/terminal-output/, evidence/screenshots/, docs/.